

Passly

Informe de la Obligatoria 1

Desarrollo de Aplicaciones II - Comision Lunes TM

Fecha de entrega: 14/09/2026

Equipo: Federico Torcini, Simon Forteza, Juan Segundo Addamo y Lucio

Version de trabajo: corte verificable del 11/09/2026

Este documento diferencia lo implementado de lo planificado. Antes de la entrega debe actualizarse con la evidencia de los componentes que se integren posteriormente al corte.

Resumen ejecutivo

Passly es una plataforma para vender y validar entradas de eventos. El backend se diseña como un monolito modular: una sola aplicacion desplegable, dividida en componentes de negocio con contratos explicitos y fronteras verificadas mediante Spring Modulith.

Al corte de este informe estan implementados los componentes Eventos y Usuarios. Ambos poseen capas de presentacion, negocio y datos, persistencia en esquemas PostgreSQL separados y contratos que evitan exponer entidades JPA. El diseño completo contempla Ventas como componente stateful y Facade, Pagos como Adapter REST y los componentes Tickets, Accesos, Notificaciones y Facturacion para las siguientes etapas.

1. Problema, alcance y arquitectura

Passly cubre el recorrido que comienza cuando un organizador publica un evento y termina cuando un validador acepta un ticket en la puerta. Un comprador selecciona entradas, realiza un pago, recibe un ticket con QR y posteriormente ese QR se marca como utilizado para impedir el doble ingreso.

La Obligatoria 1 se concentra en demostrar decisiones de arquitectura defendibles y funcionalidad incremental. La cantidad de código no reemplaza la necesidad de justificar componentes, estados, patrones, seguridad y fronteras.

1.1 Monolito modular

El backend se ejecuta como un único artefacto Spring Boot, pero no se organiza como un monolito sin límites. Cada componente publica una interfaz y mantiene sus controladores, implementaciones, entidades y repositorios bajo `internal`.

Spring Modulith analiza las dependencias y hace fallar el build ante accesos indebidos o ciclos. Esta decisión conserva transacciones locales y evita introducir sagas, fallos de red y despliegues coordinados antes de que exista una necesidad real.

La decisión completa se registra en ADR-001: Monolito modular en lugar de microservicios.

1.2 Persistencia por componente

Se utiliza una única instancia PostgreSQL con un esquema por componente. Eventos y Usuarios poseen esquemas separados. Un componente no puede consultar directamente las tablas de otro: debe utilizar su interfaz pública.

La decisión reduce complejidad operativa y mantiene visible la propiedad de los datos. El detalle se registra en ADR-002: Esquema PostgreSQL por componente.

1.3 Tecnologías principales

- Java 21 y Spring Boot 4.1.1.
- Spring Modulith 2.1.1.
- Spring Data JPA e Hibernate.
- PostgreSQL local mediante Docker Compose, sustituible por Supabase mediante configuración.
- Maven para build y pruebas.

2. Componentes y contratos

2.1 ServicioDeEventos - implementado

Eventos administra el alta y publicacion de eventos, tipos de entrada, precios, cupos y disponibilidad. Expone EventoService y DTO publicos. La implementacion de negocio, el controlador y la persistencia quedan encapsulados.

La demostracion disponible permite crear un evento, agregar tipos de entrada, publicarlo, consultar la cartelera y verificar el error 409 cuando se intenta una transicion de estado invalida.

2.2 ServicioDeUsuarios - implementado

Usuarios administra el registro y la consulta de identidades con los roles COMPRADOR, ORGANIZADOR, VALIDADOR y ADMIN. Expone UsuarioService, mientras que la entidad, el repositorio, el mapper y el encoder permanecen internos.

Las credenciales se transforman con BCrypt antes de persistirse. El hash no forma parte de UsuarioDTO y nunca se devuelve por la API. El componente incluye datos de demostracion idempotentes, uno por rol.

2.3 Componentes previstos para completar la O1

- **ServicioDeVentas:** orquesta la compra y mantiene temporalmente la seleccion del comprador. Debe implementar el estado conversacional y actuar como Facade.
- **ServicioDePagos:** oculta el contrato de una pasarela externa detras de una interfaz propia. Debe implementar el patron Adapter.
- **Seguridad:** debe autenticar credenciales y aplicar autorizacion declarativa por rol sobre al menos dos operaciones sensibles.

2.4 Componentes posteriores

Tickets emitira entradas con QR; Accesos validara su uso unico; Notificaciones enviara mensajes; Facturacion adaptara una integracion SOAP. Estos elementos pertenecen principalmente a entregas posteriores y no se presentan como implementados en este corte.

3. Capas, estado y patrones

3.1 Arquitectura en tres capas

Cada componente sigue el flujo presentacion -> negocio -> datos.

- **Presentacion:** traduce HTTP a solicitudes de negocio, valida la forma de entrada y transforma excepciones en respuestas HTTP.
- **Negocio:** contiene reglas, orquestacion y limites transaccionales.
- **Datos:** implementa persistencia mediante entidades JPA y repositorios Spring Data.

Las entidades no cruzan la frontera del componente. Los intercambios se realizan mediante contratos y DTO.

3.2 Stateful y stateless

Eventos y Usuarios son stateless: no conservan estado conversacional entre llamadas. Persistir eventos o usuarios no los vuelve stateful, porque ese estado pertenece al dominio y vive en la base.

Ventas se diseña como stateful porque debe mantener un hold temporal mientras el comprador completa el pago. Ese estado en memoria crece durante la conversacion y expira si se abandona. La implementacion debe mostrar callbacks de ciclo de vida administrados por Spring.

3.3 Patrones de diseño

DAO - implementado

Los repositorios Spring Data separan la logica de negocio del acceso a PostgreSQL. Los servicios dependen de repositorios y no escriben SQL ni administran conexiones.

Facade - planificado en Ventas

VentaService ofrecera una operacion de compra simple y ocultara la coordinacion entre disponibilidad, identidad, pago y emision. Solo debe declararse implementado cuando PAS-8 este integrado y probado.

Adapter - planificado en Pagos

PagoService presentara un contrato estable del dominio y traducira solicitudes y respuestas de la pasarela REST. Solo debe declararse implementado cuando PAS-7 este integrado y probado.

4. Seguridad, transacciones e integraciones

4.1 Estado actual de credenciales

Usuarios valida los datos de registro, rechaza emails repetidos y persiste un hash BCrypt. Este mecanismo protege la credencial almacenada, pero no constituye todavia autenticacion ni autorizacion completa.

4.2 Seguridad declarativa requerida

PAS-6 debe incorporar Spring Security, verificacion de credenciales y autorizacion mediante `@PreAuthorize`. Como minimo deben demostrarse dos operaciones sensibles, por ejemplo:

- Solo ORGANIZADOR puede crear o publicar eventos.
- Solo ADMIN puede asignar roles privilegiados.

La demostracion debe incluir tanto un acceso permitido como uno rechazado con 403. El informe final debe nombrar las operaciones realmente implementadas, sin reemplazarlas por ejemplos hipoteticos.

4.3 Transaccion critica

La confirmacion futura de una compra coordinara descuento de cupo, registro de pago y emision de tickets. Esa secuencia debe ser atomica: si falla un paso, se revierte el conjunto.

La llamada externa de facturacion queda fuera de la transaccion. Mantener una conexion a AFIP dentro de `@Transactional` bloquearia recursos y no permitiria revertir el sistema externo. La venta se confirma localmente y la factura se procesara luego mediante mensajeria con reintentos.

4.4 Integraciones previstas

- REST saliente hacia la pasarela de pago.
- Cola punto a punto orden.pagada para Facturacion.
- Topico `ticket.emitido` para Notificaciones y Accesos.
- SOAP con un servicio AFIP simulado.

5. Verificacion y demostracion

5.1 Verificacion automatica

EstructuraDeModulosTest ejecuta tres verificaciones: ausencia de violaciones entre modulos, independencia saliente de Eventos y generacion de documentacion arquitectonica. Sobre el corte del 11/09, las tres pruebas pasan con Eventos y Usuarios detectados como modulos separados.

El codigo integrado compila. El test general `contextLoads` requiere una instancia PostgreSQL accesible; por ello, la demostracion y el cierre deben ejecutarse con la infraestructura levantada. Actualmente el repositorio no posee un workflow de integracion continua.

5.2 Guion de demostracion

- 1 Levantar PostgreSQL con `docker compose up -d`.
- 2 Iniciar el backend con el perfil `demo`.
- 3 Consultar `/actuator/health` y `/actuator/modulith`.
- 4 Crear y publicar un evento.
- 5 Consultar la cartelera publica.
- 6 Registrar y consultar un usuario, verificando que el hash no aparece en la respuesta.
- 7 Ejecutar las pruebas de seguridad, Ventas y Pagos que se incorporen antes del cierre.
- 8 Ejecutar `mvnw test` y conservar el resultado como evidencia.

5.3 Pendientes para cerrar la entrega

- Integrar y probar PAS-6, PAS-7 y PAS-8.
- Actualizar este informe con clases, endpoints y resultados reales.
- Incorporar peticiones reproducibles para los nuevos flujos.
- Ejecutar la demo completa en la maquina que se utilizara en la defensa.
- Realizar una revision cruzada: cada integrante debe explicar cualquier componente.

6. Uso de inteligencia artificial y conclusiones

6.1 Declaracion de uso de IA

El equipo utilizo asistentes de inteligencia artificial como apoyo para analizar la consigna, estructurar el roadmap, contrastar alternativas de arquitectura, revisar documentacion tecnica, proponer pruebas y mejorar la redaccion.

Los asistentes no reemplazaron las decisiones del equipo. Cada sugerencia fue contrastada con la consigna, la documentacion oficial de las tecnologias y el comportamiento observado del sistema. El equipo reviso el codigo y la documentacion incorporados al repositorio y conserva la responsabilidad sobre su funcionamiento y contenido.

Antes de entregar, esta seccion debe completarse con los nombres reales de las herramientas y el uso concreto declarado por cada integrante. No deben incluirse usos que el equipo no pueda explicar o verificar.

6.2 Conclusiones

La arquitectura elegida busca equilibrio entre separacion y simplicidad. El monolito modular permite contratos claros, transacciones locales y una demostracion reproducible sin asumir prematuramente los costos de un sistema distribuido. El esquema PostgreSQL por componente refleja la misma decision en la persistencia.

Eventos y Usuarios demuestran la estructura vertical propuesta y hacen que la verificacion de Spring Modulith sea significativa. El cierre de la Obligatoria 1 depende de integrar Ventas, Pagos y Seguridad, actualizar la evidencia y ensayar el flujo completo.

Referencias

- Repositorio Passly: <https://github.com/SimonForteza/Passly>
- Spring Boot: <https://docs.spring.io/spring-boot/index.html>
- Spring Modulith: <https://docs.spring.io/spring-modulith/reference/index.html>
- PostgreSQL: <https://www.postgresql.org/docs/>
- Documentacion interna: CLAUDE.md